Absolute Cinema Ticket Booking System

Composition of the project group:

- Jarosław Gruszka 110472, pzx95639

1. Business Description

Absolute Cinema is a comprehensive IT solution for the EMS (Event Management System) class, designed to automate the full life cycle of a cinema reservation from repertoire publication to purchase finalization and ticket control at the entrance. The primary goal of the system is to maximize hall occupancy by providing an intuitive online sales channel and optimizing the workflow of cinema staff.

Context of Use:

- **B2C (Business-to-Consumer):** The system serves as a primary sales and information hub for moviegoers. Customers interact with the platform primarily via **mobile devices and web browsers** to browse the repertoire and purchase tickets on-the-go or from home.
- **Internal Operations:** Cinema staff and administrators use the system in a high-intensity environment (box office and entrance points). The context requires **high availability** and **low latency**, particularly for ticket verification via mobile scanners during peak hours before screenings.
- **Multi-Platform Accessibility:** Web Interface - For detailed administrative tasks and customer bookings.
- **Mobile Application:** Dedicated for staff to perform rapid access control.
- **Transactional Context:** Secure integration with external financial systems to ensure trust and data integrity during high-volume sale periods (e.g., blockbuster premieres).

2. System requirements specification

2.1. Functional requirements

- **Interactive Reservation Module:** The system presents a graphical, real-time hall layout to the user. It utilizes seat-mapping technology that allows for the selection of a specific seat, taking into account different zones (e.g., standard, VIP, accessible seating). The system must lock the selected seat for the duration of the transaction (e.g., 10 minutes) to prevent overbooking.
- **Dynamic Repertoire Management:** The administrator panel allows for flexible scheduling of screenings. Each film includes metadata (duration, age ratings, trailer) that automatically synchronizes with the user-facing website. The system supports multiple halls with various technical configurations (2D, 3D).
- **Pricing and Promotional Logic:** The system's calculation engine supports various tariffs (standard, discounted, senior) and discount codes. Absolute Cinema integrates with external payment gateways (e.g., PayU, BLIK, credit cards), ensuring instant transaction confirmation.
- **Ticketing System and Access Control:** After purchasing, the system generates a unique ticket in PDF format containing a mobile-scannable QR code, sent automatically to the customer's email address. A module for cinema employees (mobile application) allows for code scanning and quick verification of entry permissions for the screening.
- **Analytics and Reporting:** **The** system collects data on viewer preferences, generating financial reports (daily, monthly) and statistics regarding the most popular hours and films, providing decision-making support for the cinema's marketing management.

2.2. Non-functional requirements

- **Performance & Scalability**
  - Response Time: The graphical hall layout must load in less than 2 seconds over a standard internet connection.
  - Concurrency: The system must handle up to 1,000 simultaneous transactions without stability loss (critical during blockbuster premieres).
  - Real-time Update: Seat availability information must be refreshed in real-time (latency below 500 ms) to prevent synchronization errors.

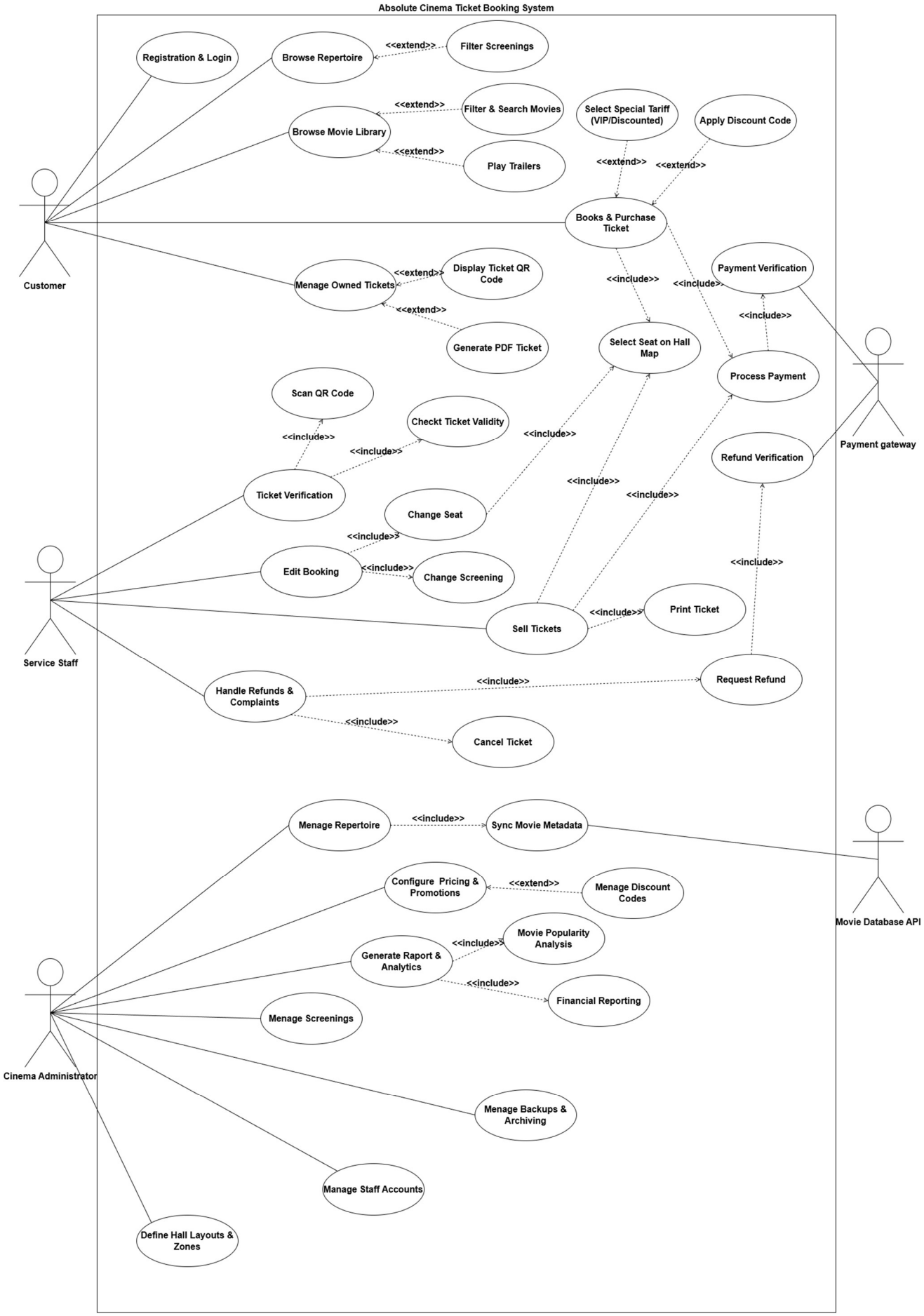
- **Security & Compliance**
  - Data Protection (GDPR): All customer personal data (e-mail, purchase history) must be encrypted and stored in compliance with GDPR regulations.
  - Payment Security: Integration with payment gateways must utilize the secure TLS 1.3 protocol and comply with PCI DSS standards.
  - Ticket Integrity: The QR code on the ticket must be digitally signed to prevent forgery or unauthorized multiple use.

- **Availability & Reliability**
  - Uptime: The system must ensure an availability level of 99.9% per year (SLA).
  - Transaction Recovery: In the event of a connection failure during payment, the system must automatically release the seat lock after exactly 10 minutes to return it to the sales pool.
  - Backup: Database backups (reservations and financial reports) must be performed every 24 hours and stored in a secure off-site location.

- **Usability**
  - Accessibility: The seat selection interface must comply with WCAG 2.1 standards, enabling keyboard navigation.
  - Mobile-First: The staff application for ticket scanning must be optimized for low-light environments (dark mode / high contrast).

- **Maintainability**
  - Metadata Sync: The automated film metadata synchronization process must not exceed 15% of the database server's CPU load during peak hours.

2.3. Use cases


Absolute Cinema Ticket Booking System
Registration & Login
Browse Repertoire
<<extend>>
Filter Screenings
Browse Movie Library
<<extend>>
Filter & Search Movies
<<extend>>
Play Trailers
Select Special Tariff (VIP/Discounted)
Apply Discount Code
<<extend>>
<<extend>>
Customer
Books & Purchase Ticket
Menage Owned Tickets
<<extend>>
Display Ticket QR Code
<<extend>>
Generate PDF Ticket
<<include>>
<<include>>
Payment Verification
<<include>>
Select Seat on Hall Map
Process Payment
Payment gateway
Scan QR Code
<<include>>
Checkt Ticket Validity
<<include>>
<<include>>
Ticket Verification
<<include>>
Change Seat
<<include>>
<<include>>
Refund Verification
<<include>>
<<include>>
Edit Booking
<<include>>
Change Screening
<<include>>
Service Staff
Sell Tickets
<<include>>
Print Ticket
Handle Refunds & Complaints
<<include>>
Request Refund
<<include>>
Cancel Ticket
Menage Repertoire
<<include>>
Sync Movie Metadata
Movie Database API
Configure Pricing & Promotions
<<extend>>
Menage Discount Codes
Movie Popularity Analysis
<<include>>
Generate Raport & Analytics
<<include>>
Financial Reporting
Cinema Administrator
Menage Screenings
Menage Backups & Archiving
Manage Staff Accounts
Define Hall Layouts & Zones

2.4. List of Actors

- **Customer:** The primary end-user interacting via the B2C interface. Their goal is to find movies, purchase tickets, and manage their bookings.
- **Service Staff:** An operational employee. They use a mobile application for ticket control and a POS (Point of Sale) system for on-site ticket sales.
- **Cinema Administrator:** A power user responsible for managing cinema configurations, movie offers, staff accounts, and financial analytics.
- **Payment Gateway:** An external actor (system) handling financial transactions and payment authorizations.
- **Movie Database API:** An external system providing movie-related data (descriptions, cast, trailers).

2.5. Functional Description of Modules

- **Customer Module:**
  - **Registration & Login:** Provides secure access to a personalized user account.
  - **Browse Screenings & Movie Library:** Allows the customer to view current screenings and the movie database. These functions are enhanced via **<<extend>>** relationships for optional filtering, searching, and playing trailers (**Play Trailers**).
  - **Book & Purchase Ticket:** The core business process.
  - **<<include>> relationships:** This process always requires selecting a seat (**Select Seat on Hall Map**) and processing the payment (**Process Payment**).
  - **<<extend>> relationships:** Customers may optionally apply discounted/VIP tariffs or use promo codes (**Apply Discount Code**).
  - **Manage Owned Tickets:** Allows customers to access purchased tickets, which can be displayed as a QR code or downloaded as a PDF file.

- **Service Staff Module:**
  - **Ticket Verification:** The entry control process. It involves scanning the QR code and automatically verifying the ticket's validity against the database.
  - **Edit Booking:** Enables staff to change seats or screenings at the customer's request (handling emergency or accidental situations).
  - **Sell Tickets:** On-site box office sales. It utilizes the same underlying mechanisms as the customer system (seat map, payment processing) but concludes with a physical ticket printout.
  - **Handle Refunds & Complaints:** Refund management. The system automatically cancels the ticket and initiates a refund through the external provider (**Request Refund**).

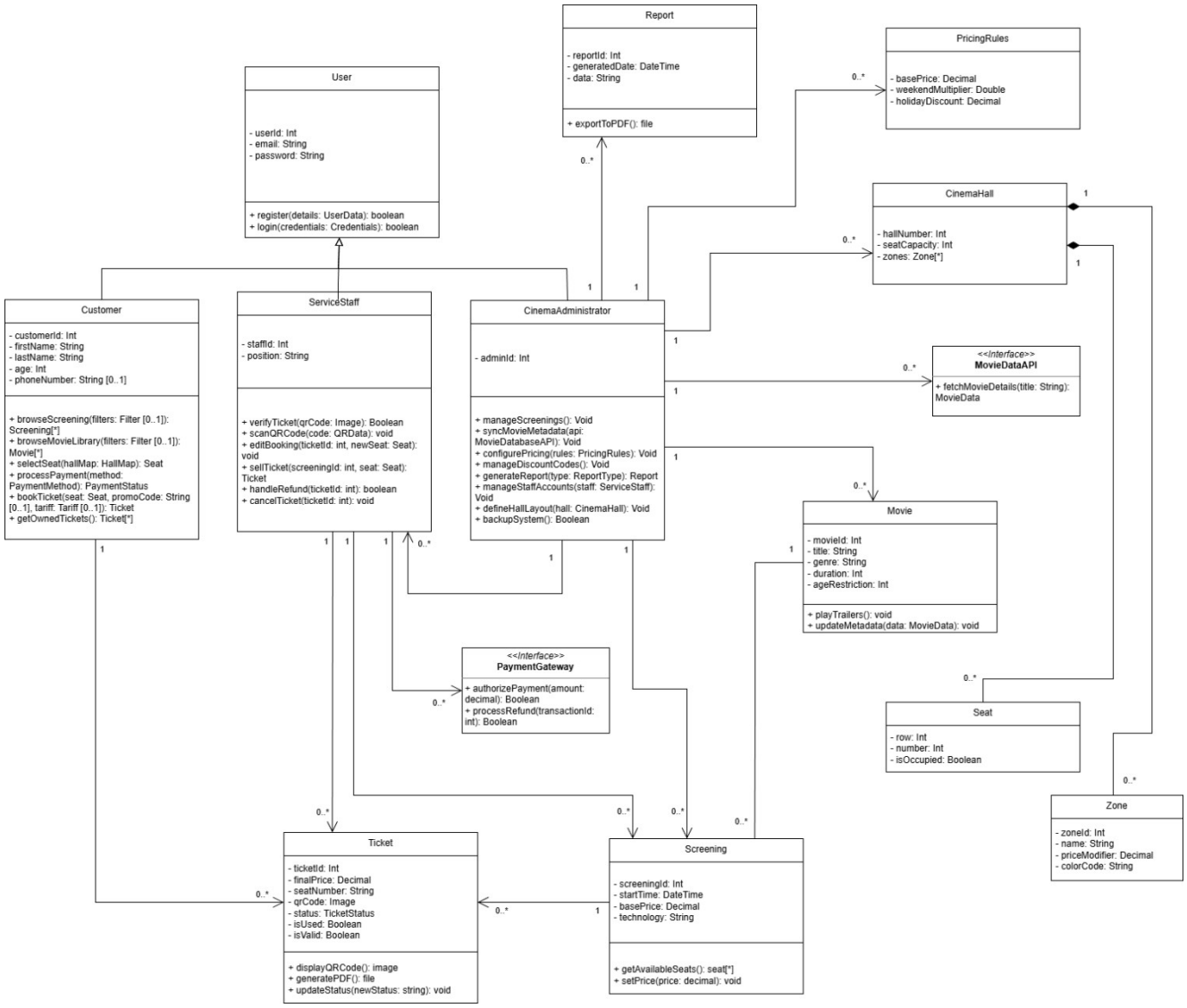
- **Administrative Module:**
  - **Manage Repertoire & Screenings:** The Administrator plans the schedule. The system automatically fetches movie data via the **Movie Database API** (**Sync Movie Metadata**).
  - **Configure Pricing & Promotions:** Management of ticket pricing tiers and discount code pools.
  - **Define Hall Layouts & Zones:** Configuration of the physical hall layout, including the definition of VIP and standard zones.
  - **Generate Report & Analytics:** A module for generating popularity reports and financial statements for management.
  - **System Maintenance:** Covers data safety (**Manage Backups & Archiving**) and management of staff credentials.

2.6. Key System Process (Include/Extend Relationships)

- **Payment Integration:** The **Process Payment** use case always includes a verification step involving the external **Payment Gateway** (**Payment Verification**).
- **Data Consistency:** Both the Customer and Staff use the same **Select Seat on Hall Map** use case. This ensures real-time synchronization and prevents double-booking through a seat-lock mechanism.
- **Optionality:** The **<<extend>>** relationships (e.g., **Apply Discount Code**) represent extension points that are not mandatory for completing a transaction, providing flexibility in the user flow.

3. System model

3.1. Class diagram

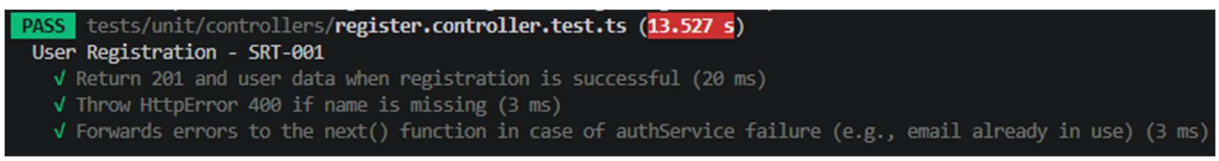


3.2. Activity diagram

...

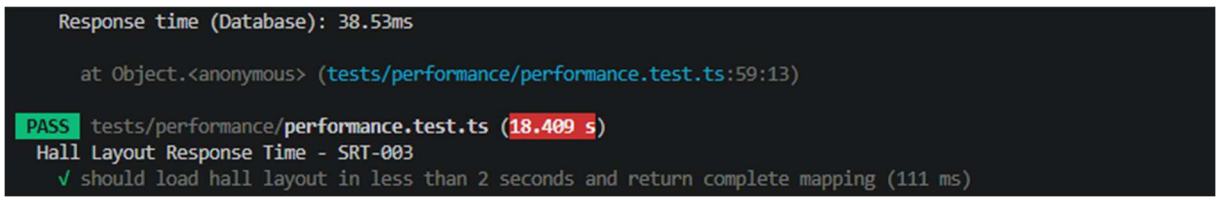
4. Test case scenarios.

Identifier: SRT-001	Name: User Registration	Author: Jaroslaw Gruszka	Date Created: 25.04.2026
Prerequisites:	• The application is running and the database is connected. • The user does not have an existing account.		
Input Data:	• username: `John Trump` • e-mail: `johnyt@test.com` • password: `js@3rasl#2d!`		
Interaction Description:	1. The client sends a POST request to the registration endpoint with user data (name, surname, email, password). 2. The AuthController triggers validation to check if all mandatory fields are present in the request body. 3. If validation passes, the controller invokes the authService.register method to handle business logic and database persistence. 4. If the service encounters an issue (e.g., the email is already taken), the controller catches the exception. 5. Any caught errors are forwarded to the global error handling middleware using the next() function.		
Expected Result:	• Successful Registration: The system returns an HTTP 201 (Created) status code along with a JSON body containing a `success` message, the new user's unique ID, and their email. • Validation Failure: If a mandatory field (such as `name`) is missing, the system throws an HttpError 400 with a specific error code (e.g., MISSING_FIELDS). • Error Propagation: In case of service-level failures (e.g., `Email already exists`), the error is correctly passed to the next() function to ensure a proper error response is sent to the client.		

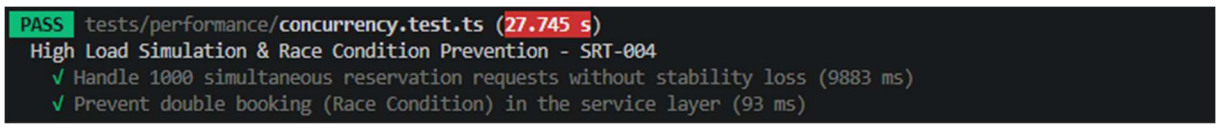


Identifier: SRT-002	Name: Ticket and Access Control Flow	Author: Jaroslaw Gruszka	Date Created: 25.04.2026
Prerequisites:	• The system is running with mocked repositories. • A valid screening and seat selection are provided.		
Input Data:	• user_id: `user_123` • screening_ID: `screening_999` • seat_id: `seat_a1` • payment_provider: `BLIK`		
Interaction Description:	1. The system calls reservationService.createReservation. 2. It generates a unique QR code and triggers the PDF/Email delivery simulation. 3. A mobile scanner (staff app) checks the ticket status in the database. 4. The system evaluates if the ticket is valid for entry.		
Expected Result:	**Regarding Step 1:** • The system successfully generates a unique QR code, PDF generation and Email delivery simulations are triggered without errors. • The system **denies** entry if the ticket status is **INACTIVE**. • The system **allows** entry if the ticket status is **ACTIVE**.		

Identifier: SRT-003	Name: Hall Layout Response Time	Author: Jaroslaw Gruszka	Date Created: 27.04.2026
Prerequisites:	• The system is running with mocked database repositories to isolate API latency. • Authenticated user session is simulated via authMiddleware bypass.		
Input Data:	• endpoint: `GET /api/user/reservations/screenings/any-id/seats` • Simulated hall size: `100 seats`. • Occupied seats: 2 (seat-5, seat-10).		
Interaction Description:	1. The test triggers a GET request to fetch the graphical hall layout for a specific screening. 2. The system calculates the availability of 100 seats by cross-referencing the seatRepository and reservationRepository. 3. performance.now() captures the high-resolution timestamp at the start and end of the request.		
Expected Result:	• The system returns a status code 200 OK. • The response body contains the complete seat mapping for the requested hall. • The total execution time (latency) is strictly less than 2000ms, fulfilling the SLA requirement specified in the business description.		



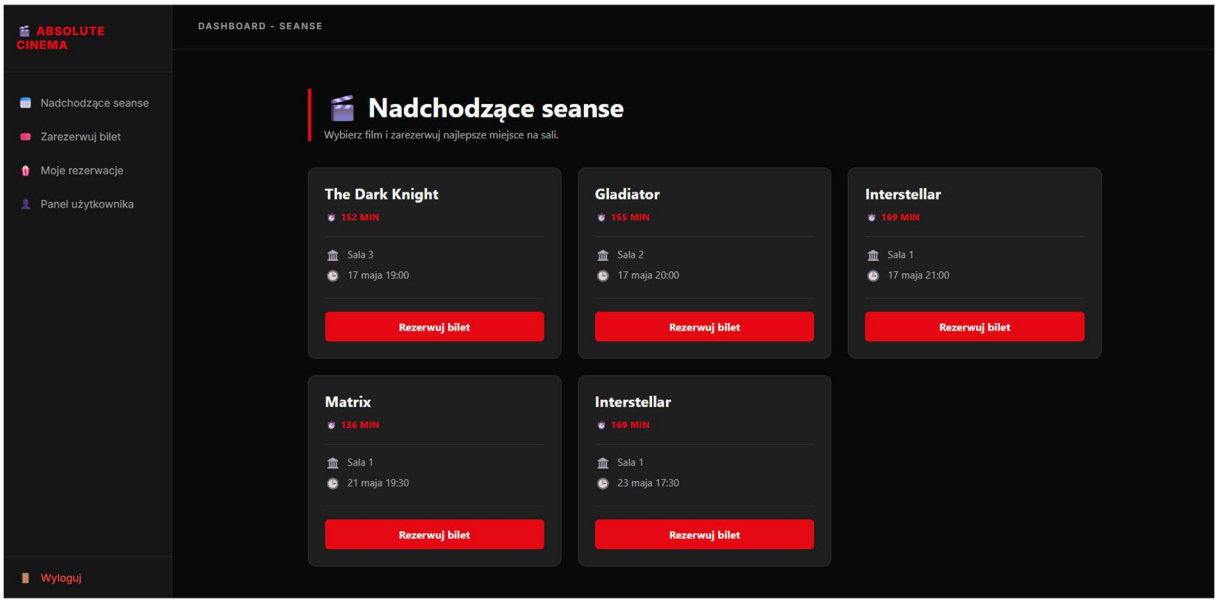
Identifier: SRT-004	Name: High Load Simulation & Race Condition Prevention	Author: Jaroslaw Gruszka	Date Created: 27.04.2026
Prerequisites:	• The system is running in a test environment with mocked database repositories. • Rate limiting is bypassed to allow high-volume traffic simulation.		
Input Data:	• 1,000 simultaneous unique reservation requests (High Load). • 10 concurrent requests for the same seat ID (Race Condition).		
Interaction Description:	1. The test environment initiates a stress test by firing 1,000 simultaneous reservation requests to evaluate system stability. 2. Multiple concurrent requests target a single specific seat to test the service layer's integrity during a potential race condition.		
Expected Result:	• Stability: The system should handle 1,000 simultaneous reservation requests without stability loss (verified performance: ~2692 ms). • Integrity: The system should prevent double booking (Race Condition) in the service layer, allowing only one successful reservation (verified performance: ~33 ms). • Reliability: All responses are processed without any unhandled exceptions or service crashes.		



5. Graphic design

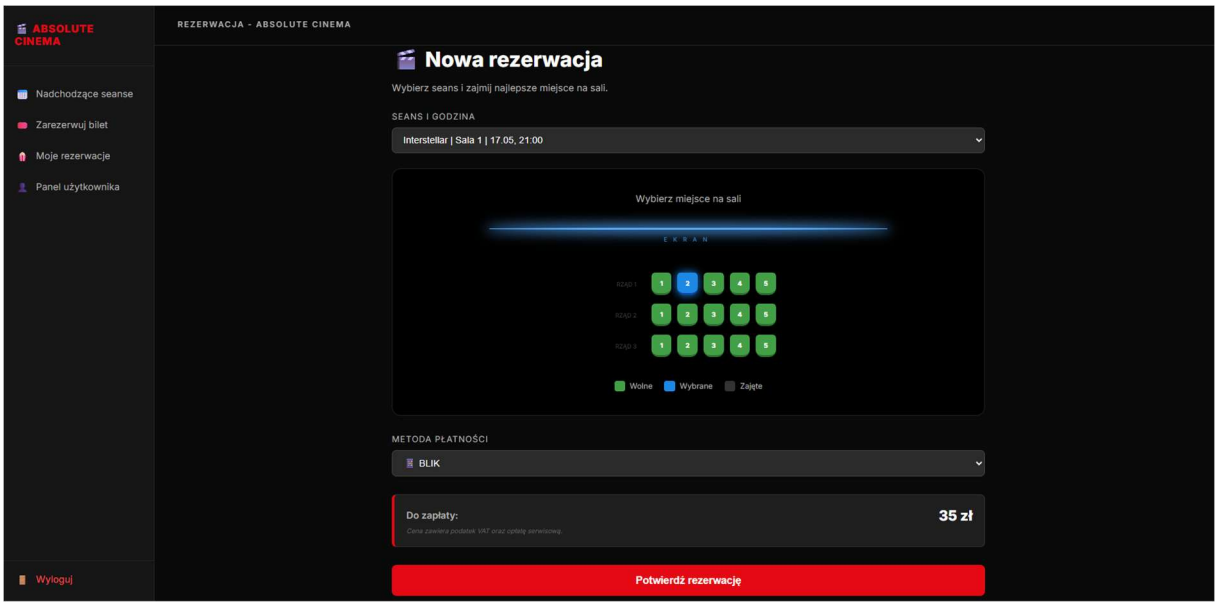
User (customer) Interface

Customer Home Page – Repertoire Browser



The user interface displaying the list of available movies as clean poster cards. The screen allows customers to browse basic movie details and proceed directly to ticket purchasing by clicking on a selected showtime.

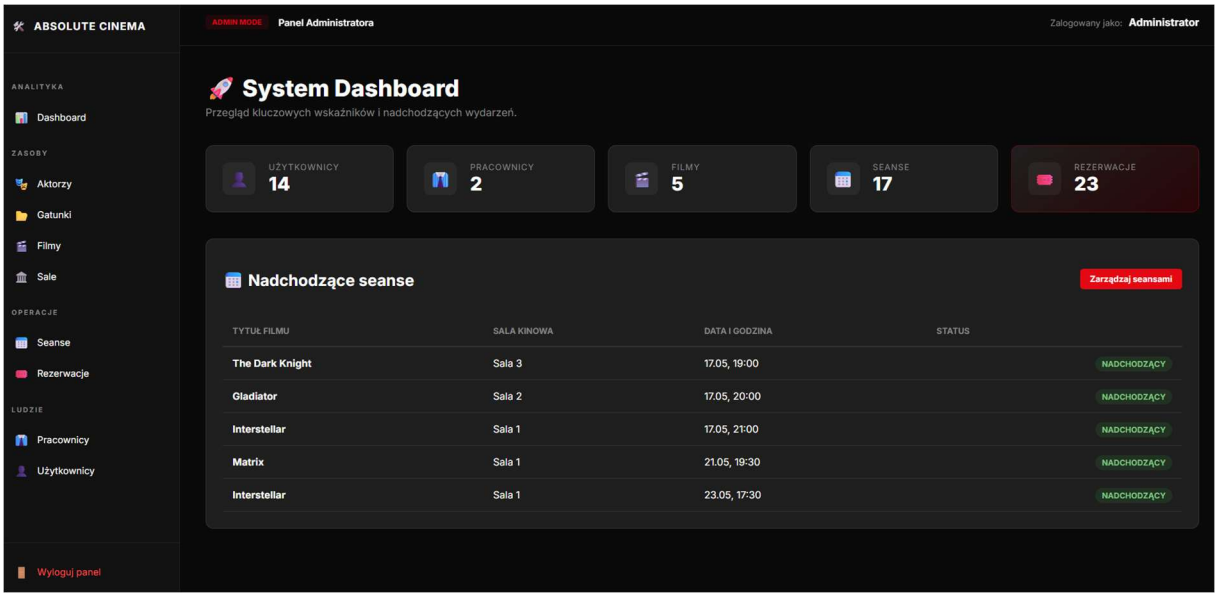
Interface for dynamic seat selection on the hall layout



The movie details page enabling the customer to read the production description and interactively select available seats before finalizing the reservation.

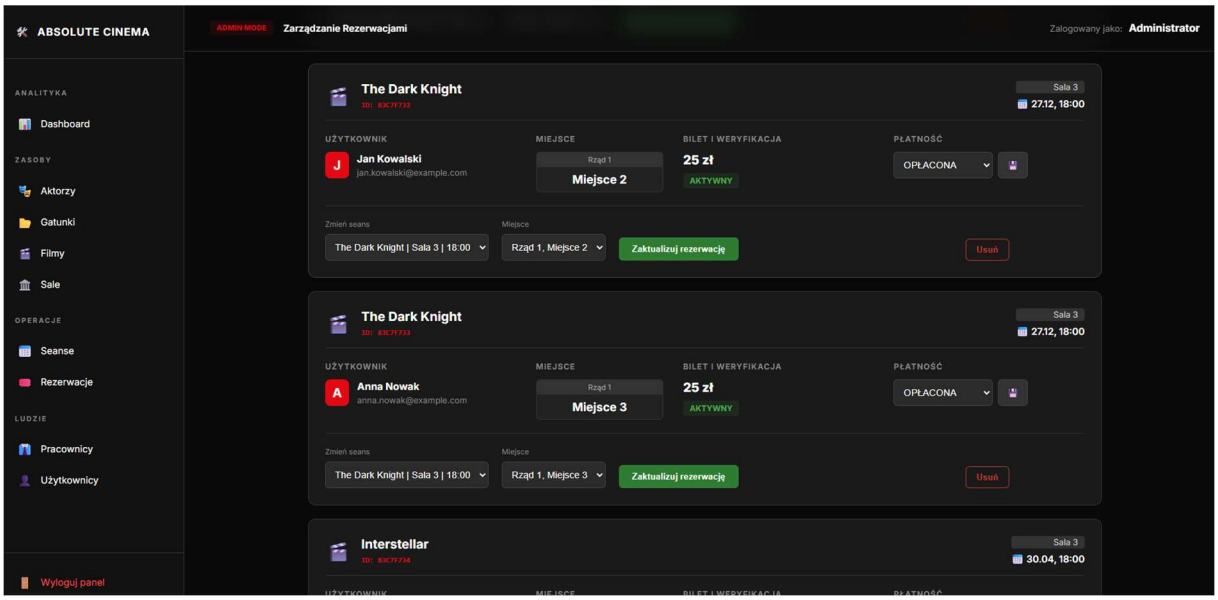
Administrator Panel Interface

Administrator Panel – System Dashboard



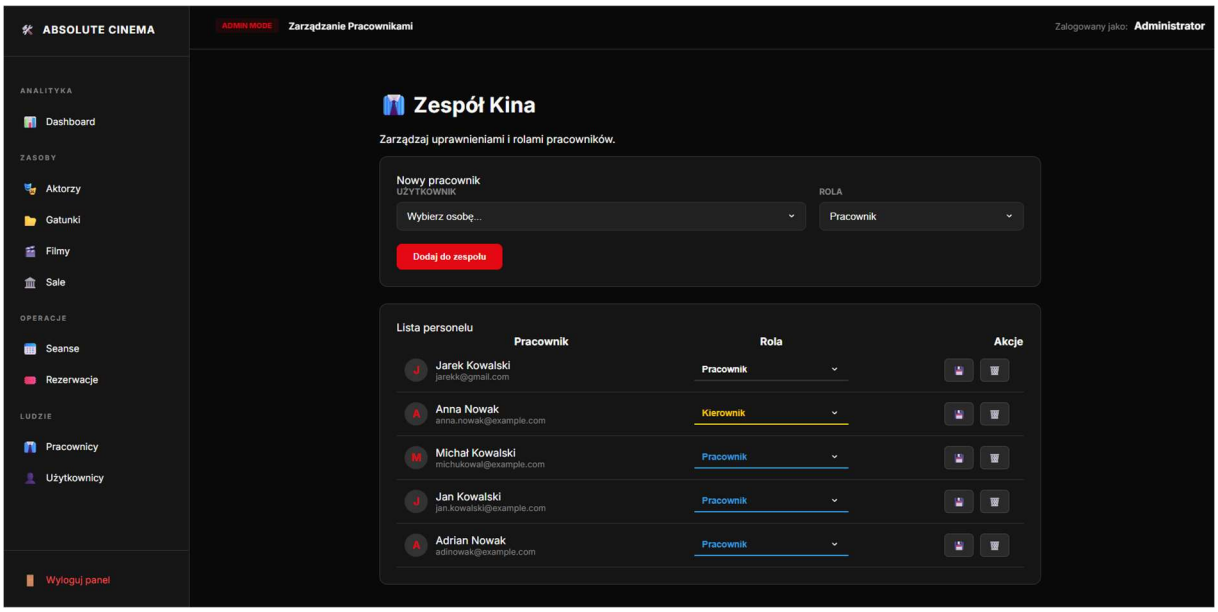
The main dashboard view for the Absolute Cinema system administrator. The screen displays a summary of platform statistics (users, movies, screenings, bookings) and a list of upcoming movie projections across individual halls, providing a quick overview of the cinema's operational status and direct access to schedule management.

Administrator Panel – Booking List and Editing



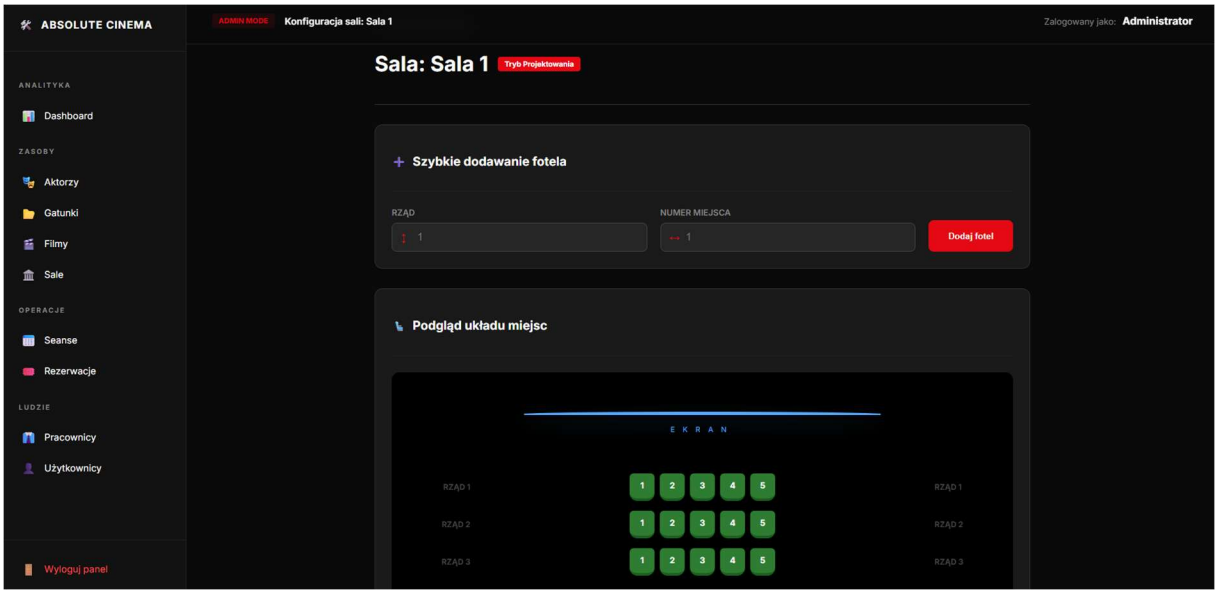
The operational module of the system designed for reviewing and modifying user reservations. The interface allows cinema staff to view transaction details, manually reassign seats or showtimes for specific customers, manage payment statuses, and cancel reservations.

Administrator Panel – Staff Permissions Management



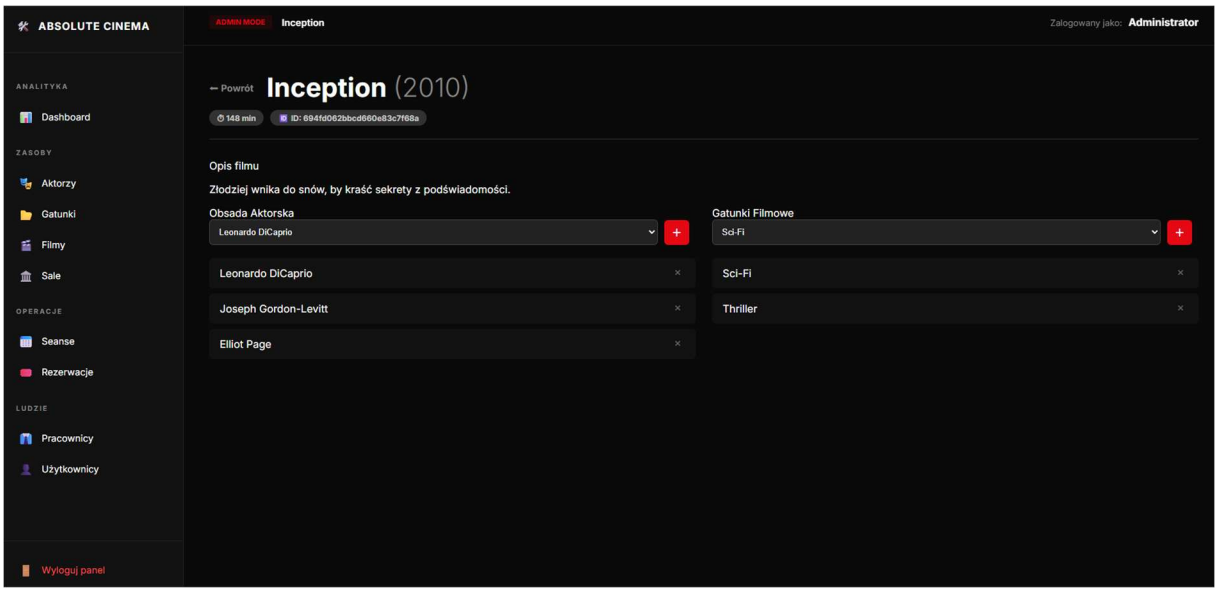
The administrative module view dedicated to granting system privileges and managing cinema staff accounts. The interface enables quick onboarding of new team members, assigning them relevant business roles (Manager / Employee) directly within the table, and removing personnel from the system.

Cinema hall structure editing view



Administrative interface used for interactively adding seats and previewing the physical layout of the chairs in a selected Absolute Cinema hall.

Interface for editing the cinema's movie repository



Administrative panel designed for inserting new movie productions into the system and reviewing the current database of titles.

The system also includes simplified panels for adding genres and actors, allowing them to be linked to movies within the database.

6. System implementation

6.1. Technology Stack

The backend of the Absolute Cinema system was implemented using the Node.js runtime environment along with the Express.js framework. Due to strict stability and reliability requirements, TypeScript was selected as the programming language. It provides static typing, thereby minimizing the risk of runtime errors.

The key technologies and libraries utilized in the project include:

- **Database:** The system utilizes a non-relational MongoDB database, which is ideal for storing complex, semi-structured data such as hall layouts and movie metadata. Communication with the database is handled via the Mongoose (Object Data Modeling) library, which enforces strict Schemas and provides data validation.
- **Security & Compliance:** Route protection and session security (e.g., for administrator and staff roles) are implemented using JSON Web Tokens (JWT). User passwords are securely hashed using the bcrypt library. Furthermore, the application employs the helmet package to secure HTTP headers, as well as express-rate-limit to protect against Brute Force attacks and sustain high performance under heavy traffic loads.
- **Ticket Generation:** In compliance with the Access Control Flow requirements, the qrcode library was integrated to dynamically generate scannable codes on tickets.

6.2. System Architecture & Design Patterns

The application was built following the N-Tier Architecture paradigm, which ensures a clear Separation of Concerns and provides the scalability required for EMS-class systems:

- **Controllers (API Interface Layer):** Responsible for handling incoming HTTP requests (e.g., from the web application or staff mobile scanners), performing initial request validation, and delegating execution to the appropriate services. They utilize centralized error handling via a custom exception class (HttpError).
- **Services (Business Logic Layer):** The core of the system where business processes are encapsulated. For instance, the ReservationService class aggregates the logic for payment creation, ticket (QR) generation, and seat reservations, ensuring the atomicity of the process.
- **Repositories (Data Access Layer - Repository Pattern):** Acts as an abstraction layer over Mongoose. Isolating database queries (e.g., reservationRepository, ticketRepository) allows for straightforward database mocking during testing (e.g., scenario SRT-002) and ensures a seamless potential migration to other database systems in the future.
- **Models (Data Layer):** Defines entity models using Mongoose (User, Screening, Reservation, Ticket, Seat), incorporating mechanisms such as Soft Delete (via the is_active field).

6.3. Concurrency Control & Data Integrity

In a cinema reservation system (particularly during blockbuster premieres), preventing "double-booking" (Race Conditions), as described in scenario SRT-004, is critical. The system tackles this challenge on two levels:

- **Database Level:** Unique Partial Indexes are applied. The reservation collection maintains a unique index on the screening_id and seats_id pair, enforced exclusively for active reservations (is_active: true). This physically prevents duplicate entries from being written to the database.
- **Business Logic Level:** A pseudo-transactional mechanism featuring a rollback function inside a try...catch block is implemented within the service layer (ReservationService). If the reservation process encounters a failure at any stage (e.g., an external payment gateway error), the system automatically deletes the pre-generated ticket and payment record, releasing the seat lock.

6.4. Performance & SLA Fulfillment

According to the non-functional specification, the system must process data in real-time with minimal latency:

- **Hall Layout Load Time (SRT-003):** By isolating queries and optimizing connections within the Repositories, the endpoint fetching the graphical hall layout (GET /api/user/reservations/screenings/.../seats) maps seats and reservations (for a sample size of 100 seats) well within the required 2-second threshold, often executing in under a few dozen milliseconds in test environments.
- **Traffic Spike Resilience (SRT-004):** Performance benchmarks demonstrated that the asynchronous nature of Node.js, combined with proper process isolation, allows the API to process a high-load simulation of up to 1,000 simultaneous concurrent requests without any stability loss.

6.5. Quality Assurance and Testing

A rigorous testing strategy was established using the Jest framework alongside the Supertest library for End-to-End (E2E) API integration testing. All defined use cases are thoroughly covered by the test suite:

- **Unit Tests:** Implemented for components such as registration controllers (scenario SRT-001), verifying validation error handling and error propagation via the middleware next() function.
- **Integration Tests:** Comprehensive validation of core business flows (scenario SRT-002), proving the reliability of QR code generation and ticket status transitions required for the staff mobile app (Ticket and Access Control Flow).
- **Performance and Stress Tests:** Dedicated test modules (performance.test.ts, concurrency.test.ts) successfully demonstrate the resilience of the system's architecture, meeting the most demanding criteria specified in the project scope.